

UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI INGEGNERIA

Corso di Laurea Magistrale in Bioingegneria — LM-21

Implementazione di un comportamento di Trail Following bioispirato mediante Spiking Neural Network

Ricreare il comportamento di una formica in un robot simulato

Tesina per l'esame di Robotica Bioispirata

Scritto da: Claudio Giacobbe

Anno accademico 2025/2026

Abstract

Il presente elaborato descrive la progettazione e la valutazione di un sistema di robotica bioispirata capace di riprodurre, in ambiente simulato, il comportamento di trail following osservabile nelle formiche. L'idea biologica viene trasformata in una catena artificiale composta da percezione visiva, elaborazione, decisione e movimento. La percezione è ottenuta mediante una camera simulata in CoppeliaSim; OpenCV viene utilizzato come sistema teacher per individuare la traccia rossa e generare automaticamente le etichette del dataset. La decisione finale viene invece affidata a una Spiking Neural Network (SNN) realizzata con PyTorch e snnTorch.

La rete è shallow e fully connected: l'immagine RGB viene ridotta a 32×32 pixel, trasformata in 3072 caratteristiche e codificata in spike mediante rate coding su 20 timestep. Due blocchi Linear-LIF producono cinque classi corrispondenti alle posizioni FAR_LEFT, LEFT, CENTER, RIGHT e FAR_RIGHT. Durante l'inferenza la distribuzione di probabilità sulle classi viene convertita in una posizione laterale continua, successivamente filtrata e trasformata in un comando di rotazione per le ruote.

La presentazione di progetto riporta, su cinque prove, un success rate medio del 96,93%. L'analisi del CSV di valutazione allegato aggiunge una lettura più dettagliata di una singola prova di circa 60 s: 521 frame, confidence media 0,909 e 8,70 FPS calcolati sul file. I risultati confermano la fattibilità del paradigma, ma mostrano anche che la metrica "fuori pista" è una stima basata sull'immagine e non una misura geometrica in coordinate del mondo.

Indice

1. Bioispirazione e formulazione del problema
2. Obiettivo e metodologia
3. Ambiente di simulazione e architettura del sistema
4. Percezione visiva: OpenCV come baseline e teacher
5. Dataset e definizione delle classi
6. Spiking Neural Networks e neuroni LIF
7. Architettura, codifica e addestramento
8. Inferenza e controllo del robot
9. Valutazione sperimentale e risultati
10. Discussione critica
11. Conclusioni e sviluppi futuri

Appendice A. Struttura software e parametri principali

Bibliografia e materiali di progetto

1. Bioispirazione e formulazione del problema

La robotica bioispirata parte dall'osservazione di sistemi biologici e cerca di trasferirne principi di comportamento, organizzazione o calcolo in sistemi artificiali. Nel progetto considerato l'ispirazione è il comportamento di trail following delle formiche: l'animale deve percepire una traccia, interpretarne la posizione rispetto al proprio movimento e modificare la traiettoria per mantenerne il contatto.

Nel sistema artificiale il feromone non è modellato fisicamente. La traccia è rappresentata da un trail rosso presente nella scena di CoppeliaSim. La camera del robot osserva la scena e produce un'immagine RGB. Il problema può quindi essere espresso come una trasformazione percettivo-motoria: a partire dall'immagine bisogna ottenere una decisione di sterzata.

La presentazione del progetto formalizza questo passaggio attraverso la catena percezione → elaborazione → decisione → movimento. Nel caso biologico la formica percepisce la traccia, elabora lo stimolo, modifica il proprio comportamento e mantiene la traiettoria; nel caso robotico la camera sostituisce il sistema percettivo, l'elaborazione software sostituisce i processi decisionali e il comando alle ruote produce il movimento.

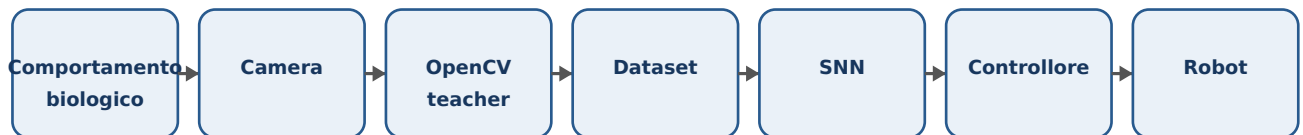


Figura 1 — Trasposizione concettuale dal comportamento biologico alla pipeline robotica.

La scelta è interessante per un esame di Robotica Bioispirata perché non consiste semplicemente nel realizzare un controllo visivo: l'obiettivo è verificare se un comportamento relativamente semplice può essere appreso da una rete neurale a eventi e poi riprodotto dal robot. La SNN non riceve direttamente le regole geometriche usate dal teacher durante l'inferenza; apprende invece una relazione tra immagini RGB e posizione laterale del percorso.

2. Obiettivo e metodologia

L'obiettivo del progetto è ricreare in un robot simulato il comportamento di trail following utilizzando una Spiking Neural Network. La strategia progettuale separa volutamente due fasi: una fase di supervisione, nella quale OpenCV individua la traccia e assegna una label, e una fase di inferenza, nella quale la SNN deve produrre la decisione senza utilizzare OpenCV come componente decisionale.

Il README del progetto esplicita la pipeline: camera di CoppeliaSim → immagine RGB → SNN snnTorch → cinque classi → controllore delle ruote. La raccolta del dataset utilizza invece l'algoritmo OpenCV esistente come teacher. Questa separazione è fondamentale: OpenCV serve a creare il bersaglio dell'apprendimento, mentre la SNN costituisce il modello che deve sostituire la decisione di sterzata basata direttamente sul contorno.

Fase	Input	Elaborazione	Output
Percezione	Camera CoppeliaSim	Acquisizione RGB	Immagine
Supervisione	Immagine RGB	HSV + maschera + contorno	Posizione del trail
Dataset	Immagine + posizione	Discretizzazione	5 classi
Training	Immagini + label	Rate coding + LIF + backpropagation	Modello SNN
Inferenza	Nuova immagine RGB	SNN su 20 timestep	Classe + posizione continua
Controllo	Posizione filtrata	Guadagno + saturazione	Velocità ruote

Questa impostazione rende il progetto interpretabile anche dal punto di vista sperimentale: il comportamento del teacher costituisce il riferimento da cui nasce il dataset, mentre il comportamento della SNN può essere valutato in termini di stabilità, errore laterale, confidence, sterzata e capacità di mantenere il percorso.

Un aspetto metodologico importante è che non bisogna confondere la qualità del teacher con la capacità generale della SNN. Se il dataset è costruito mediante regole basate sul colore rosso e sul contorno, la rete apprende una rappresentazione di quel comportamento specifico. Il passaggio a scenari più complessi richiederebbe quindi dati e supervisione più ricchi.

3. Ambiente di simulazione e architettura del sistema

Il robot viene simulato in CoppeliaSim e controllato da Python attraverso la ZeroMQ Remote API. Il progetto richiede una camera con nome /camera e quattro giunti di ruota: /rollingJoint_fl, /rollingJoint_rl, /rollingJoint_rr e /rollingJoint_fr. Il software mantiene la convenzione delle ruote del progetto originale.

Il codice di acquisizione utilizza `sim.getVisionSensorImg()`. L'immagine restituita da CoppeliaSim viene interpretata come RGB, convertita in array NumPy e ribaltata verticalmente per compensare la convenzione dell'immagine. La scena viene aggiornata durante la simulazione senza richiedere l'uso di `sim.handleVisionSensor()`.

Dal punto di vista funzionale il sistema comprende quattro blocchi: ambiente simulato, percezione, rete neurale e controllo. CoppeliaSim fornisce la dinamica e la scena; la camera produce i dati; Python gestisce la comunicazione e il preprocessing; `snnTorch` implementa la rete; infine il controllore traduce la posizione stimata in velocità dei quattro giunti.



Rate coding → 20 timestep → dinamica temporale dei neuroni LIF

Figura 2 — Struttura della SNN utilizzata nel progetto.

L'architettura complessiva è volutamente semplice. L'obiettivo non è costruire una rete profonda, ma verificare il principio secondo cui un comportamento motorio può essere trasferito a una SNN shallow fully connected. La semplicità facilita inoltre l'analisi dei parametri e il collegamento tra l'architettura mostrata nella presentazione e il codice effettivamente fornito.

3.1 Ambiente di simulazione: CoppeliaSim

L'ambiente sperimentale è realizzato in CoppeliaSim 4.10, utilizzato come simulatore dell'interazione tra robot, sensori e ambiente. La scena è costruita attorno a un grande elemento Floor, sul quale è presente una rappresentazione testurizzata di un ambiente urbano: nell'immagine della scena si riconoscono una superficie stradale, edifici lungo il perimetro, vegetazione e una rotatoria centrale. Il robot BM_Bot opera sulla superficie del Floor e osserva il percorso attraverso una camera montata sul modello.

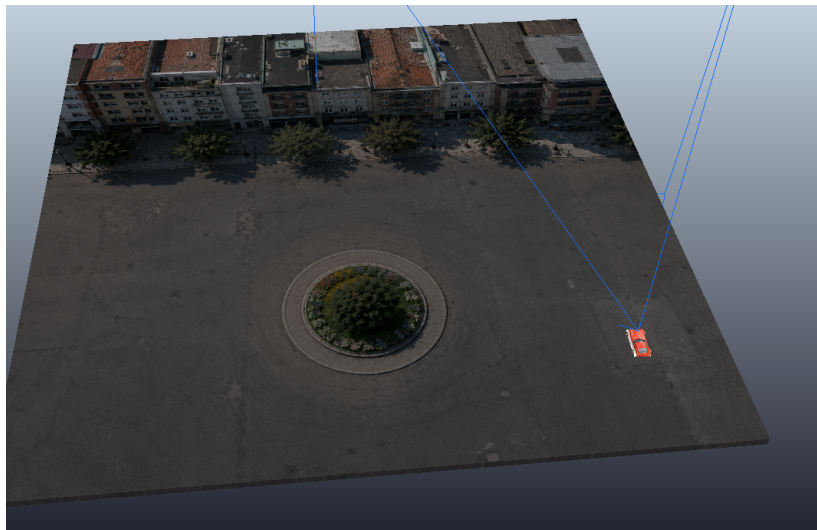


Figura 10 — Vista dell'ambiente simulato in CoppeliaSim e del BM_Bot sulla superficie testurizzata.

La Scene Hierarchy mostra la struttura logica dell'ambiente. Il nodo Floor contiene il percorso originale path e il relativo child script; BM_Bot contiene quattro giunti di ruota (rollingJoint_fl, rollingJoint_fr, rollingJoint_rr, rollingJoint_rl), la base BM_Bot_base e la camera. Sono inoltre presenti gli oggetti di servizio della scena, tra cui DefaultCamera, XYZCameraProxy e DefaultLights.

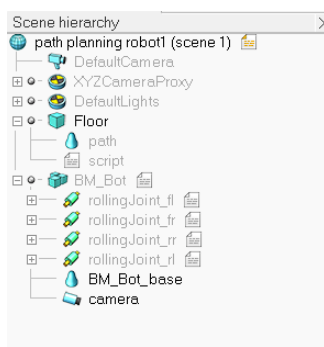


Figura 11 — Gerarchia della scena utilizzata per il progetto.

Un elemento particolarmente importante per l'esperimento è il child script non-threaded associato al Floor. Lo script utilizza l'API sim e, all'avvio, recupera i riferimenti a /Floor, /BM_Bot e /Floor/path. Il percorso originale viene reso invisibile, ma non cancellato; successivamente viene generato un nuovo percorso rosso chiuso a partire dalla posizione attuale del robot.

Elemento	Funzione nell'esperimento
/Floor	Superficie dell'ambiente e riferimento del sistema di coordinate.
/Floor/path	Percorso originale, nascosto durante la generazione casuale.
/Floor child script	Generazione e rigenerazione automatica del trail rosso.
/BM_Bot	Robot mobile utilizzato per il trail following.
rollingJoint_*	Quattro attuatori delle ruote.
/BM_Bot/camera	Sensore visivo utilizzato dalla pipeline SNN.

Il generatore di percorso rende l'esperimento meno dipendente da una singola geometria. Lo script cerca un'ellisse di dimensioni prossime a quelle del Floor, la deforma mediante armoniche casuali e costruisce 96 punti; il bordo viene poi discretizzato in segmenti rossi statici e non-responder. La larghezza della striscia è 0,28 m. Il primo punto viene forzato esattamente sulla posizione del robot e l'ultimo segmento viene collegato al primo, ottenendo un loop chiuso.

4. Percezione visiva: OpenCV come baseline e teacher

Il primo sistema di controllo del progetto è basato su visione artificiale convenzionale. L'immagine RGB viene convertita nello spazio HSV, più adatto a isolare un colore specifico. Il rosso viene gestito mediante due intervalli di tonalità, $H=0...15$ e $H=165...179$, perché il rosso è presente alle due estremità della scala Hue.

Le due maschere vengono unite e sottoposte a operazioni morfologiche di apertura e chiusura con un kernel 3×3 . L'apertura riduce il rumore locale mentre la chiusura tende a ricomporre regioni vicine. Successivamente vengono estratti i contorni esterni e viene selezionato quello di area maggiore, purché superi una soglia minima di 20 pixel.

Dal rettangolo delimitatore del contorno si ricavano le coordinate del centro, c_x e c_y . Il codice del teacher usa inoltre un punto di riferimento posto al centro del bordo inferiore dell'immagine. L'errore angolare è calcolato come:

$$\omega = \arctan((x_{\text{ref}} - c_x) / (y_{\text{ref}} - c_y))$$

Il valore angolare viene poi usato dal controllore originale per modulare velocità di avanzamento e intensità della rotazione. Nella raccolta del dataset questo controllo mantiene il robot in movimento mentre il teacher registra immagini e label. In particolare, il dataset non contiene immagini generate artificialmente: le immagini vengono acquisite dalla camera della simulazione.

Il ruolo di OpenCV è quindi duplice ma temporalmente separato: è una baseline per il comportamento originale e diventa un teacher durante la generazione delle label. Nel test SNN, invece, OpenCV non viene utilizzato per decidere la sterzata; la sua presenza nella valutazione è indipendente e serve come proxy visivo per misurare la posizione della pista.

5. Dataset e definizione delle classi

Il programma di raccolta del dataset è configurato per acquisire fino a 3000 campioni. Ogni campione contiene un'immagine RGB e una riga nel file labels.csv con nome del file, label, c_x , c_y e area del contorno. Se il teacher non trova il percorso, il robot viene fermato e il campione non viene salvato.

La posizione orizzontale del centro del percorso viene normalizzata rispetto alla larghezza dell'immagine. Le soglie sono 20%, 40%, 60% e 80%. Si ottengono cinque classi:

Classe	Intervallo normalizzato	Significato
0	< 0,20	FAR_LEFT
1	0,20 - 0,40	LEFT
2	0,40 - 0,60	CENTER
3	0,60 - 0,80	RIGHT
4	> 0,80	FAR_RIGHT

La scelta di cinque categorie rappresenta un compromesso tra semplicità e informatività. Una singola decisione binaria "sinistra/destra" sarebbe troppo povera per descrivere la gradualità della correzione, mentre un numero molto elevato di classi renderebbe più difficile il training. Le cinque categorie consentono alla SNN di apprendere una discretizzazione della posizione e, in fase di inferenza, di recuperare una stima continua usando le probabilità di classe.

Il training usa inoltre uno WeightedRandomSampler per ridurre gli effetti di eventuali squilibri tra classi. Il codice calcola pesi inversamente proporzionali alla frequenza delle classi nel sottoinsieme di training e costruisce il DataLoader a partire da tali pesi.

Il dataset è quindi il punto di passaggio tra comportamento deterministico e comportamento appreso: il teacher fornisce la supervisione, ma la rete viene addestrata sulle immagini RGB, non sulle maschere HSV. Questa distinzione è centrale per interpretare correttamente l'architettura.

6. Spiking Neural Networks e neuroni LIF

Le Spiking Neural Networks sono reti neurali nelle quali l'informazione viene rappresentata mediante eventi discreti, gli spike. Nel progetto questa scelta ha un duplice valore: da un lato richiama la comunicazione neuronale biologica, dall'altro introduce esplicitamente una dimensione temporale nel processo computazionale.

A differenza di una rete convenzionale che può produrre un'attivazione continua a partire da un singolo input, la SNN evolve su più timesteps. Il modello implementato con `snnTorch` usa neuroni Leaky Integrate-and-Fire (LIF). Il neurone accumula informazione nel potenziale di membrana, produce uno spike quando raggiunge la soglia e applica un meccanismo di reset.

Nel codice il parametro di decadimento del neurone è $\beta=0,9$. Entrambi i blocchi LIF usano un surrogate gradient di tipo fast sigmoid e un reset di tipo "subtract". Il surrogate gradient è necessario perché la funzione di spike è discreta e non presenta un gradiente utile nella forma classica; durante il backpropagation viene quindi utilizzata un'approssimazione differenziabile.

Nel progetto la bioispirazione non è dunque limitata al comportamento della formica. È presente anche a livello computazionale: il modello cerca di rappresentare l'informazione attraverso dinamiche temporali simili, in forma astratta, alla comunicazione neuronale.

È importante mantenere una distinzione tra potenziale e spike. Nel training e nell'inferenza il codice conserva la sequenza dei potenziali di membrana dei neuroni di output; l'ultimo potenziale viene utilizzato come score per le cinque classi. Gli spike costituiscono la dinamica interna, mentre il potenziale finale fornisce il valore utilizzato per la classificazione.

Il progetto non misura il consumo energetico e non esegue il modello su hardware neuromorfico. Pertanto, la possibile efficienza energetica delle SNN va considerata come prospettiva, non come risultato sperimentale ottenuto in questo lavoro.

7. Architettura, codifica e addestramento

La rete PathSNN riceve immagini RGB ridimensionate a 32×32 pixel. Le tre componenti cromatiche producono quindi $32 \times 32 \times 3 = 3072$ feature. L'architettura è composta da un primo livello Linear $3072 \rightarrow 128$, seguito da LIF, e da un secondo Linear $128 \rightarrow 5$, seguito da un secondo LIF.



Rate coding → 20 timestep → dinamica temporale dei neuroni LIF

Figura 3 — Architettura effettiva implementata in `snn_model.py`.

Prima di essere presentato ai neuroni, l'input viene normalizzato nell'intervallo $[0,1]$. La funzione `spikegen.rate()` di `snnTorch` converte l'intensità di ogni elemento in una sequenza temporale di spike: il valore del pixel determina la probabilità/frequenza con cui l'elemento contribuisce allo spike train. Il numero di timestep è 20.

L'addestramento è supervisionato. Il codice usa `CrossEntropyLoss` sui potenziali di membrana e somma la loss sui 20 timestep, dividendola infine per il numero di timestep. L'ottimizzatore è Adam con learning rate 10^{-3} . Il batch size è 32, il numero di epoche è 20 e il validation split è 20%. Il seed impostato è 42 per favorire la riproducibilità.

Parametro	Valore
Input	32×32 RGB = 3072 feature
Hidden layer	128 neuroni
Output	5 classi
β LIF	0,9
Timestep	20
Batch size	32
Epoche	20
Learning rate	10^{-3}
Optimizer	Adam
Validation split	20%

7.1 Funzione di loss e logica di training

Il ciclo di training esegue per ogni batch un forward completo sui 20 timestep. Per ciascun istante t viene calcolata la Cross Entropy tra il potenziale di membrana dei cinque neuroni di output e la label. La loss complessiva è la media delle loss temporali:

$$L = (1/T) \sum_{t=1}^T CE(V(t), y)$$

Questa scelta obbliga la rete a costruire una rappresentazione temporale utile non soltanto nell'ultimo istante. Il potenziale di membrana viene quindi trattato come una traccia temporale che accumula informazione. Il gradiente viene propagato attraverso la dinamica dei LIF grazie al surrogate gradient.

Durante la validazione il modello viene posto in modalità evaluation e non vengono aggiornati i pesi. La previsione della classe è ottenuta tramite `argmax` del potenziale finale. Il checkpoint migliore viene salvato quando la validation accuracy supera il miglior valore precedente. Nel file di modello vengono conservati anche i parametri necessari a ricostruire l'architettura, tra cui input size, hidden size, numero di output, β , image size e numero di timestep.

Questa procedura produce il file `path_snn.pth`, che viene poi caricato dal programma di controllo. In questo modo la fase di training è separata dall'esecuzione robotica: il robot in fase di inferenza non deve riaddestrare la rete, ma utilizza il checkpoint già prodotto.

Dal punto di vista metodologico, la pipeline è quindi: raccolta → preprocessing → training → selezione del miglior checkpoint → inferenza → controllo → valutazione. Questa separazione rende il progetto ripetibile e permette di sostituire singoli moduli senza modificare l'intero sistema.

8. Inferenza e controllo del robot

Durante l'inferenza la camera produce una nuova immagine RGB. Il programma la ridimensiona alla dimensione richiesta dal checkpoint, la normalizza e la appiattisce. La SNN la processa su 20 timestep. Il potenziale finale dei cinque neuroni di output viene trasformato in probabilità mediante softmax.

Le cinque classi vengono associate alle posizioni laterali discrete -1 , $-0,5$, 0 , $+0,5$ e $+1$. Invece di utilizzare solamente la classe con probabilità massima, il controllore calcola la media pesata:

$$\text{posizione} = \sum p_i \cdot \text{posizione}_i$$

Questa soluzione trasforma l'output discreto della classificazione in un valore continuo. Se, per esempio, due classi adiacenti hanno probabilità comparabili, la posizione risultante cade tra le due e il controllo può correggere la traiettoria in modo meno brusco.

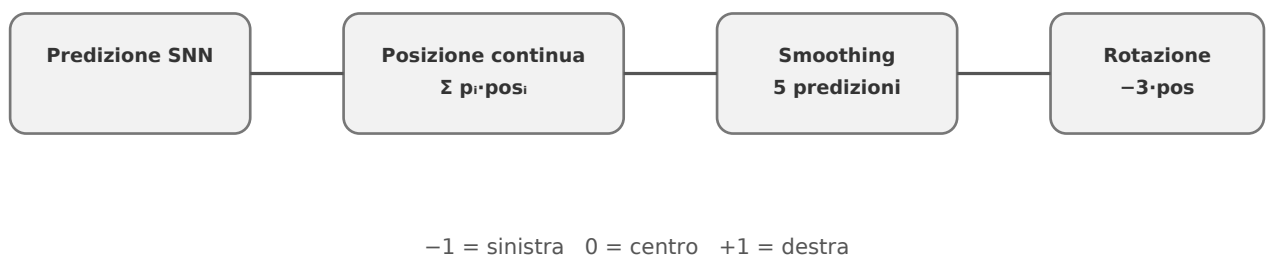


Figura 4 — Conversione della decisione SNN in comando di sterzata.

La posizione viene ulteriormente filtrata con una finestra di cinque predizioni. Il valore filtrato viene moltiplicato per un guadagno di rotazione pari a $3,0$ e limitato a $\pm 4,5$. La velocità di avanzamento nominale è $3,5$, ma viene ridotta quando l'errore laterale cresce: il fattore di velocità è almeno $0,45$ e diminuisce in funzione del valore assoluto della posizione.

Il controllore usa quindi una combinazione di decisione neurale e legge di controllo relativamente semplice. La SNN decide dove si trova il percorso; il controllo traduce quella stima in una rotazione continua delle ruote.

8.1 Dal comando di rotazione alle ruote

Il movimento del robot è implementato tramite le quattro rollingjoint. Il comando generico contiene tre componenti: avanzamento, movimento laterale e rotazione. Nel progetto SNN la componente laterale è impostata a zero, mentre la rotazione è quella derivata dalla posizione filtrata.

Ruota	Velocità target
rollingjoint_fl	–forward – lateral – rotation
rollingjoint_rl	–forward + lateral – rotation
rollingjoint_rr	–forward – lateral + rotation
rollingjoint_fr	–forward + lateral + rotation

La convenzione dei segni viene mantenuta coerente con il progetto originale. Il programma SNN imposta quindi il comando con forward negativo secondo la convenzione delle ruote, lateral=0 e rotation derivata dalla rete. La saturazione impedisce alla rotazione di superare $\pm 4,5$.

Un elemento significativo è il legame tra errore laterale e velocità: quando la rete riconosce una posizione fortemente laterale, il robot riduce la velocità di avanzamento per lasciare maggiore margine temporale alla correzione. Quando la posizione è vicina al centro, il fattore di velocità tende a 1 e il robot procede alla velocità nominale.

Questa strategia introduce una forma semplice di comportamento adattivo: la rete non decide direttamente ogni singola velocità delle ruote, ma fornisce una variabile di stato percettiva continua che alimenta un controllore. È una scelta utile perché mantiene separati il problema di percezione/decisione e quello di attuazione.

Il programma gestisce inoltre condizioni di errore: se la camera non restituisce un'immagine valida, il robot viene fermato. Alla terminazione del programma, anche i comandi alle ruote vengono azzerati e la simulazione viene arrestata.

9. Valutazione sperimentale

La presentazione di progetto riporta una valutazione su cinque run, con un success rate medio del 96,93% e deviazione standard 2,33 punti percentuali. Le altre metriche aggregate riportate sono errore laterale medio 0,187, tempo di completamento 61,52 s, uscite dal trail 3,07%, tempo medio di recupero 1,16 s e frequenza di elaborazione 9,48 FPS.

Metrica	Media	Dev. std.	Range riportato
Success rate (%)	96,93	2,33	95,81-99,81
Errore laterale medio e	0,187	0,014	0,167-0,206
Tempo (s)	61,52	0,16	61,39-61,78
Fuori pista (%)	3,07	2,11	0,19-6,19
Recupero (s)	1,16	0,55	0,14-1,58
Computazione (FPS)	9,48	0,60	8,48-10,15

Questi valori sono quelli riportati nella presentazione fornita per l'esame e costituiscono il quadro sperimentale aggregato del progetto. Non devono essere interpretati come misure di consumo energetico o come prestazioni di un robot reale: l'esperimento è svolto nel simulatore.

Il codice di valutazione allegato utilizza anche metriche più granulari. In particolare, calcola la confidence media e minima, l'errore laterale SNN, l'errore della pista stimato dalla camera, la percentuale di frame considerati potenzialmente fuori pista, il tempo massimo di uscita continua e i cambi di direzione dello sterzo.

Il codice specifica esplicitamente che "fuori pista" è una stima basata sull'immagine: non è una distanza geometrica reale in metri perché non viene definito un riferimento world-space. Questa precisazione è metodologicamente importante e viene ripresa nella discussione.

9.1 Analisi del CSV di valutazione allegato

Oltre ai risultati aggregati della presentazione, è stato analizzato direttamente il file `snn_evaluation.csv` fornito insieme al progetto. Il file contiene 521 osservazioni e copre circa 59,90 s. La pista risulta visibile in tutti i frame secondo il campo `camera_track_visible`.

Indicatore	Valore calcolato dal CSV
Frame analizzati	521
Durata coperta dal CSV	59,90 s
FPS medio (frame/durata)	8,70
Confidence media	0,909
Confidence minima	0,500
Errore SNN medio assoluto	0,108
Errore SNN massimo assoluto	0,497
Errore camera medio assoluto	0,180
Errore camera massimo assoluto	0,551
Frame oltre soglia fuori pista $ e > 0,55$	19,19%
Correzione sterzo media assoluta	0,324
Correzione sterzo massima	1,491

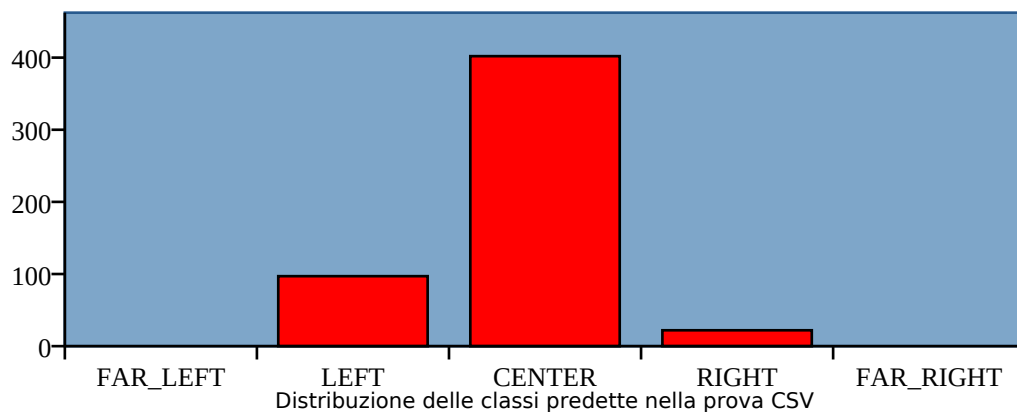


Figura 5 — Distribuzione delle classi predette nella prova presente nel CSV.

Nel CSV compaiono tre classi: CENTER in 402 frame, LEFT in 97 frame e RIGHT in 22 frame; FAR_LEFT e FAR_RIGHT non compaiono nella prova analizzata. La rete mostra quindi un comportamento prevalentemente centrato, con correzioni moderate verso sinistra e destra.

9.2 Dinamica temporale della predizione

La dinamica della posizione stimata mostra come la SNN non produca solamente decisioni isolate, ma una sequenza temporale di valori che viene poi filtrata. Nel CSV la posizione assume valori compresi tra circa $-0,484$ e $+0,497$, con media complessiva negativa pari a circa $-0,066$ prima di considerare ulteriori interpretazioni del percorso.

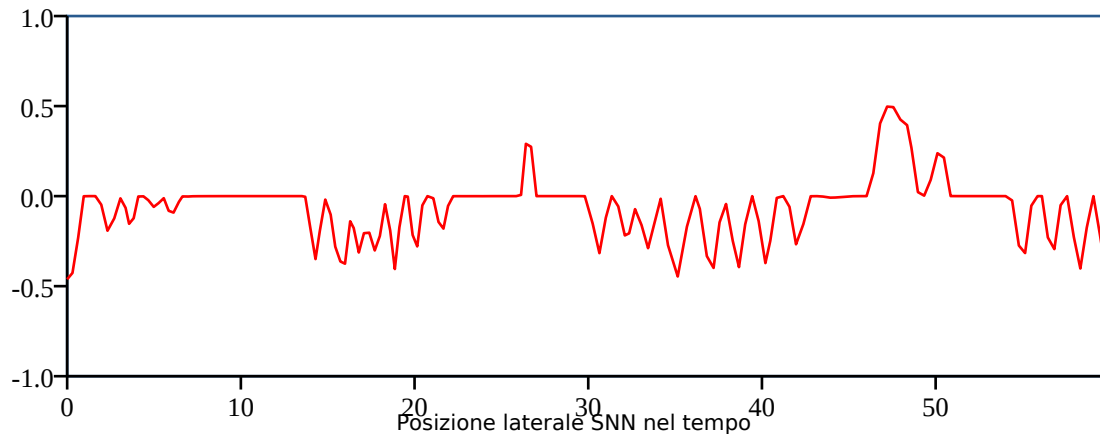


Figura 6 — Posizione continua stimata dalla SNN durante la prova allegata.

La confidence media è 0,909, mentre la minima è circa 0,500. La prova non presenta frame sotto la soglia di 0,50 usata dal codice di valutazione. Inoltre, nessun frame supera la soglia $|posizione| > 0,75$ definita come errore SNN molto elevato.

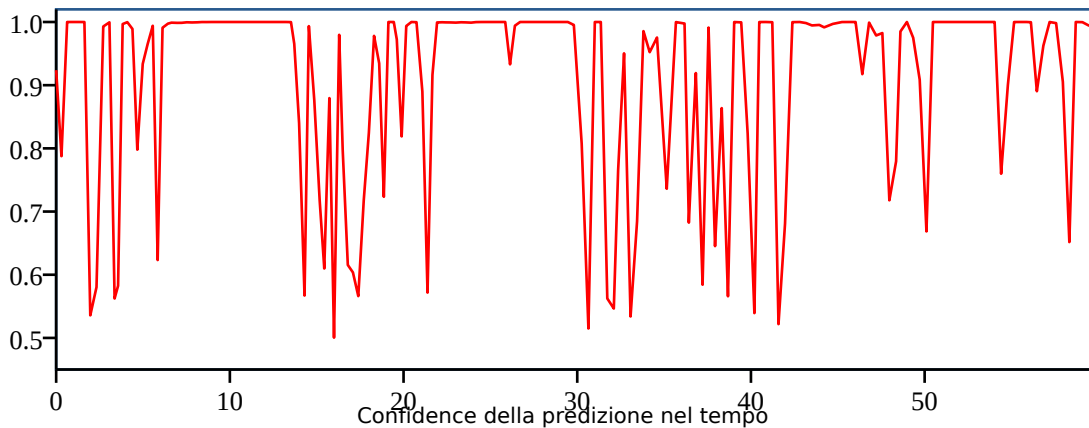


Figura 7 — Confidence della classe predetta nel tempo.

Questi risultati suggeriscono una predizione relativamente stabile nel caso specifico del CSV. Tuttavia, la confidence non deve essere interpretata automaticamente come misura di correttezza: è la sicurezza interna della classificazione, mentre la correttezza richiede un riferimento esterno, come il teacher o una misura geometrica indipendente.

9.3 Errore visivo e controllo

Il valutatore contiene una seconda elaborazione OpenCV indipendente dalla SNN. La funzione di detection cerca la pista rossa e calcola un errore normalizzato rispetto al centro dell'immagine. Viene data maggiore importanza alla parte bassa dell'immagine, ritenuta più utile per stimare dove passa la pista davanti al robot.

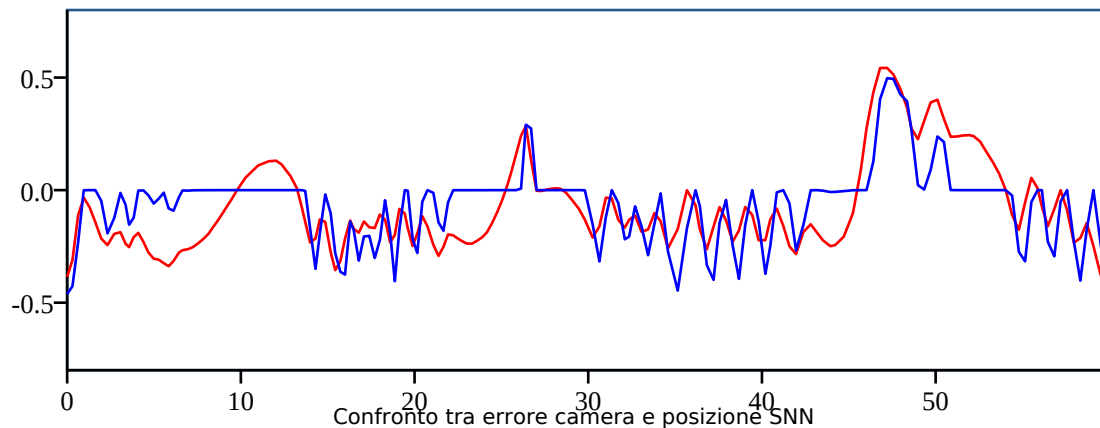


Figura 8 — Confronto qualitativo tra proxy camera e posizione SNN.

Nel CSV l'errore camera medio assoluto è 0,180 e il massimo è 0,551. La soglia di potenziale uscita pista utilizzata dal valutatore è 0,55; la quota di frame oltre tale soglia è circa 19,19%. Questo valore non coincide con il 3,07% di "fuori pista" riportato nella presentazione perché i due risultati appartengono a valutazioni diverse e, soprattutto, utilizzano procedure di aggregazione differenti.

La correzione di sterzo media assoluta nel CSV è 0,324, con massimo 1,491. Il codice applica un guadagno di 3,0 alla posizione filtrata e una saturazione $\pm 4,5$. La rotazione osservata resta quindi molto al di sotto del limite massimo durante la prova analizzata.

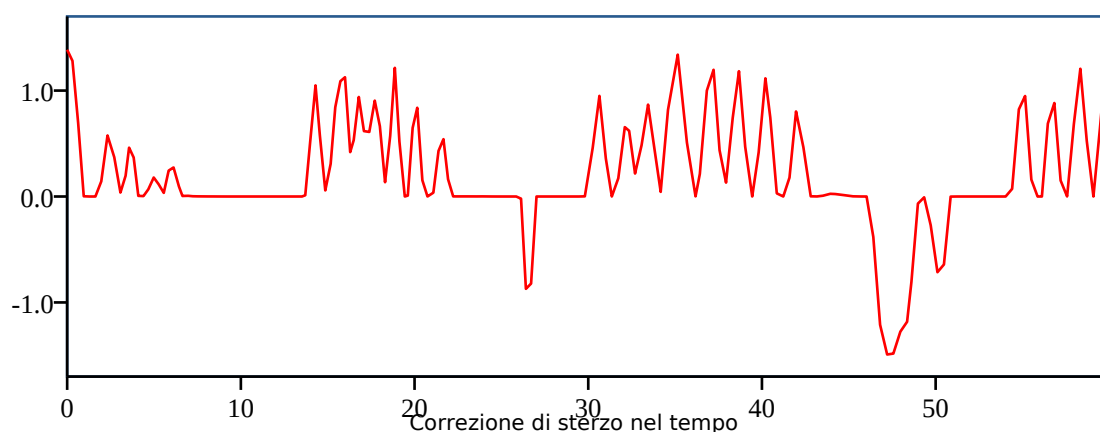


Figura 9 — Comando di rotazione prodotto dal controllore.

10. Discussione critica

Il primo risultato importante è la coerenza tra la struttura concettuale presentata e l'implementazione. La pipeline "OpenCV teacher → dataset → SNN" è effettivamente realizzata: il teacher assegna le classi, la SNN riceve immagini RGB, il modello evolve su 20 timestep e il controllore utilizza la distribuzione di probabilità per ottenere una posizione continua.

Un secondo elemento positivo è la separazione dei livelli. La SNN non deve imparare direttamente le quattro velocità delle ruote: deve apprendere una variabile intermedia, cioè la posizione laterale del trail. Il controllore traduce poi tale variabile in movimento. Questa scelta riduce la complessità del problema di apprendimento e rende più semplice analizzare il comportamento.

Esistono però limiti importanti. Il trail è artificiale e cromaticamente molto definito; il teacher OpenCV dipende dal colore rosso e dal contorno maggiore; il dataset è costruito nello stesso ambiente in cui il robot viene testato; la rete è shallow e completamente connessa, quindi non sfrutta strutture spaziali tipiche delle immagini come farebbe una CNN.

Anche la valutazione richiede cautela. Il success rate medio del 96,93% riportato nella presentazione è un risultato incoraggiante, ma il significato preciso della metrica deve essere mantenuto distinto dalle metriche del CSV. Il valutatore allegato dichiara infatti che la condizione "fuori pista" è una stima basata sulla camera e non una distanza geometrica reale.

Infine, la frequenza di circa 9,48 FPS riportata nelle cinque run, e 8,70 FPS nel CSV analizzato, indica che il sistema è ancora lontano da un'implementazione real-time ad alta frequenza. Per un prototipo simulato è un dato utile; per un robot reale sarebbe necessario considerare hardware, latenza della camera, latenza della rete e frequenza del controllo in modo integrato.

10.1 OpenCV e SNN: due ruoli differenti

Aspetto	OpenCV	SNN
Input	Camera RGB	Camera RGB
Elaborazione	HSV + maschera + contorno	Rate coding + LIF
Training	Non necessario	Supervisionato
Label	Produce le label	Riceve le label
Temporalità	Elaborazione frame-based	20 timestep
Output	Posizione del contorno	5 classi + posizione continua
Ruolo nel sistema	Teacher / riferimento	Controller percettivo appreso
Bioispirazione	Comportamentale	Comportamentale + computazionale

Il confronto evidenzia che OpenCV e SNN non sono due implementazioni equivalenti dello stesso modulo. OpenCV costituisce il meccanismo che rende possibile la supervisione automatica; la SNN è il componente che deve apprendere a riprodurre il comportamento.

Questo schema è particolarmente adatto a un progetto didattico perché permette di costruire un dataset senza etichettatura manuale. Il teacher genera una forma di “dimostrazione” artificiale del comportamento. La rete cerca quindi di effettuare una sorta di behavior cloning semplificato: osserva lo stesso tipo di immagine che vedrà durante il controllo e impara quale decisione di posizione è associata.

Il limite è che la rete eredita i limiti del teacher. Se la regola OpenCV produce una label errata o non generalizza a una nuova condizione visiva, anche il dataset contiene quell'errore. In prospettiva, la qualità del sistema potrebbe essere migliorata utilizzando più condizioni di illuminazione, trail differenti, dati più vari e una supervisione che includa informazioni geometriche o comportamentali più ricche.

11. Conclusioni e sviluppi futuri

Il progetto ha mostrato come un comportamento bioispirato relativamente semplice possa essere tradotto in una pipeline robotica composta da percezione, apprendimento e controllo. Il comportamento di trail following viene rappresentato mediante una traccia artificiale e un robot simulato in CoppeliaSim. OpenCV fornisce la supervisione automatica, mentre una SNN shallow fully connected apprende a classificare la posizione laterale del trail.

La rete utilizza immagini RGB 32×32 , rate coding su 20 timestep e neuroni LIF con $\beta=0,9$. L'output è costituito da cinque classi, ma la fase di controllo sfrutta la distribuzione di probabilità per ottenere una posizione continua. Una finestra di smoothing di cinque predizioni e una legge di controllo con guadagno 3,0 trasformano tale posizione in una rotazione delle ruote.

La valutazione aggregata riportata nella presentazione raggiunge un success rate medio del 96,93% su cinque run. Il CSV allegato, relativo a una prova di circa 60 s, mostra confidence media 0,909, errore laterale SNN medio assoluto 0,108 e 8,70 FPS calcolati direttamente sui 521 campioni. Questi risultati sostengono la fattibilità del sistema, pur con le cautele metodologiche discusse.

Gli sviluppi futuri indicati dal progetto possono essere organizzati in cinque direzioni. La prima è il passaggio dalla traccia artificiale alla guida autonoma, ad esempio riconoscimento della strada e mantenimento della corsia. La seconda è l'utilizzo di event-based vision ed event camera, per ridurre la dipendenza da immagini frame-based. La terza è il trasferimento su robot reale, che richiederebbe affrontare rumore sensoriale, latenza e variabilità dell'ambiente.

Una quarta direzione è l'esecuzione su hardware neuromorfico, coerente con il paradigma event-driven delle SNN. La quinta riguarda l'apprendimento autonomo, con online learning o reinforcement learning, e l'estensione a swarm robotics, dove più agenti bioispirati possono coordinarsi e produrre comportamenti collettivi.

In conclusione, il valore principale del progetto non è soltanto il raggiungimento di una percentuale di successo elevata, ma la dimostrazione di una catena concettuale coerente: un comportamento biologico viene astratto, trasformato in un problema di percezione e decisione, appreso da una rete spiking e infine tradotto in movimento robotico.

Appendice A — Struttura software, materiali e riferimenti

I materiali forniti per il progetto comprendono la presentazione, gli script Python per simulazione, raccolta dati, modello SNN, training, controllo e valutazione, il file CSV di valutazione, il modello addestrato e la scena Coppeliasim. La struttura software descritta nel README è la seguente:

File / componente	Funzione
task_zmq_curve.py	Baseline OpenCV e controllo originale
collect_snn_dataset.py	Acquisizione immagini e generazione label
snn_model.py	Definizione della PathSNN
train_snn.py	Training e selezione del checkpoint
run_snn_robot.py	Inferenza SNN e controllo del robot
evaluate_snn.py	Metriche e salvataggio CSV
path_snn.pth	Checkpoint del modello addestrato
snn_evaluation.csv	Misure temporali di una prova
Coppeliasim scene	Ambiente simulato, camera e ruote

Materiali di progetto utilizzati per la redazione

- Presentazione “Presentazione-progettorobotica-claudiogiacobbe(2).pptx”, utilizzata come riferimento principale per obiettivi, architettura, metodologia, risultati aggregati e sviluppi futuri.
- README del progetto “SNN PATH FOLLOWING - Coppeliasim + snnTorch”, utilizzato per verificare la pipeline software, il ruolo del teacher e i parametri principali.
- Codici Python allegati: task_zmq_curve.py, collect_snn_dataset.py, snn_model.py, train_snn.py, run_snn_robot.py ed evaluate_snn.py.
- File “snn_evaluation.csv”, analizzato direttamente per la sezione sperimentale relativa alla singola prova.
- File requirements.txt, dal quale risultano le dipendenze numpy, opencv-python, torch, snntorch e